```sql
--1. How many pizzas were ordered?
select count(*) as total_pizza_ordered
from ppd_customer_orders
```

	total_pizza_ordered bigint
1	14

```sql
--2. How many unique customer orders were made?
select count(distinct order_id) as unique_customer_order
from ppd_customer_orders;
```

	unique_customer_order bigint
1	10

```sql
--3.How many successful orders were delivered by each runner?
select runner_id, count(*) as successful_orders
from ppd_runner_orders
where cancellation is null
group by runner_id;
```

	runner_id integer	successful_orders bigint
1	1	4
2	2	3
3	3	1

```sql
--4. How many of each type of pizza was delivered?
with pizza_delivered as (
    select pizza_id,count(*) as count_of_pizza_delivered
    from ppd_customer_orders
    where order_id in (
        select order_id
        from ppd_runner_orders
        where cancellation is null
    )
    group by pizza_id
)
select pn.pizza_id,pizza_name, count_of_pizza_delivered
from pizza_names pn
left join pizza_delivered pd
on pn.pizza_id = pd.pizza_id
```

	pizza_id integer	pizza_name text	count_of_pizza_delivered bigint
1	1	Meatlovers	9
2	2	Vegetarian	3

```sql
--5. How many Vegetarian and Meatlovers were ordered by each customer?
with count_tbl as (
    select customer_id,pizza_id,count(*) as no_of_pizza_ordered
    from ppd_customer_orders
    group by customer_id,pizza_id
    order by customer_id,pizza_id
),
customers as (
    select distinct customer_id
    from ppd_customer_orders
),
cus_pizza as (
    select *
    from customers
    cross join pizza_names
    order by customer_id,pizza_id
)
select cp.customer_id, cp.pizza_id, cp.pizza_name,coalesce(ctbl.no_of_pizza_ordered,0) as no_of_pizza_ordered
from cus_pizza cp
left join count_tbl ctbl
on cp.pizza_id = ctbl.pizza_id and cp.customer_id = ctbl.customer_id
order by cp.customer_id,cp.pizza_id
```

	customer_id integer	pizza_id integer	pizza_name text	no_of_pizza_ordered bigint
1	101	1	Meatlovers	2
2	101	2	Vegetarian	1
3	102	1	Meatlovers	2
4	102	2	Vegetarian	1
5	103	1	Meatlovers	3
6	103	2	Vegetarian	1
7	104	1	Meatlovers	3
8	104	2	Vegetarian	0
9	105	1	Meatlovers	0
10	105	2	Vegetarian	1

```sql
--6. What was the maximum number of pizzas delivered in a single order?

select order_id, count(*) as no_of_pizzas
from ppd_customer_orders
where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
group by order_id
order by no_of_pizzas DESC
FETCH FIRST 1 ROW ONLY;
```

	order_id integer	no_of_pizzas bigint
1	4	3

```sql
--7. For each customer, how many delivered pizzas had at least 1 change and how many had no changes?

with changes as (
    select *,
    CASE
        when exclusions is null and extras is null then 1
        else 0
    end as no_changes,
    CASE
        when exclusions is not null or extras is not null then 1
        else 0
    end as atleast_one_change
    from ppd_customer_orders
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
    order by order_id
)
select customer_id,sum(no_changes) as no_changes, sum(atleast_one_change) as atleast_one_change
from changes
group by customer_id;
```

	customer_id integer	no_changes bigint	atleast_one_change bigint
1	101	2	0
2	103	0	3
3	104	1	2
4	105	0	1
5	102	3	0

```sql
--8. How many pizzas were delivered that had both exclusions and extras?

select
    SUM(
        CASE
            when exclusions is not null and extras is not null then 1
            else 0
        end
    ) as no_of_pizza_delivered_having_both_exclusions_and_extras
    from ppd_customer_orders
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null);
```

	no_of_pizza_delivered_having_both_exclusions_and_extras bigint
1	1

```sql
--9. What was the total volume of pizzas ordered for each hour of the day?

with hr_of_day as (
    select *,EXTRACT(hours from order_time) as hour_of_the_day
    from ppd_customer_orders
)
select hour_of_the_day, count(*) as volume_of_pizzas_ordered
from hr_of_day
group by hour_of_the_day;
```

	hour_of_the_day numeric	volume_of_pizzas_ordered bigint
1	18	3
2	21	3
3	23	3
4	13	3
5	19	1
6	11	1

```sql
--10. What was the volume of orders for each day of the week?

with dow as (
    select *,EXTRACT(isodow from order_time) as day_of_week
    from ppd_customer_orders
)
select day_of_week,count(*) as volume_of_orders
from dow
group by day_of_week
```

	day_of_week numeric	volume_of_orders bigint
1	3	5
2	4	3
3	6	5
4	5	1

Runner and Customer Experience

```sql
--1. How many runners signed up for each 1 week period? (i.e. week starts 2021-01-01)

SELECT (DATE '2021-01-01' + (((registration_date - DATE '2021-01-01') / 7)::int) * 7) AS week_start,
    count(*) as sign_ups_per_week
from runners
group by week_start
order by week_start;
```

	week_start date	sign_ups_per_week bigint
1	2021-01-01	2
2	2021-01-08	1
3	2021-01-15	1

```sql
--2. What was the average time in minutes it took for each runner to arrive at the Pizza Runner HQ to pickup the order?

with successfull_orders as (
    select DISTINCT order_id, order_time
    from ppd_customer_orders
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
),
time_diff as (
    select so.order_id,so.order_time,runner_id,pickup_time,AGE(pickup_time,order_time) as pickup_order_diff
    from successfull_orders so
    left join ppd_runner_orders pro
    on so.order_id = pro.order_id
)
select runner_id, EXTRACT(MINUTE FROM AVG(pickup_order_diff)) as AVG_TIME_IN_MINUTE_BTW_order_and_pickup
from time_diff
group by runner_id;
```

	runner_id integer	avg_time_in_minute_btw_order_and_pickup numeric
1	3	10
2	2	20
3	1	14

```sql
--3. Is there any relationship between the number of pizzas and how long the order takes to prepare?

with successfull_orders as (
    select DISTINCT order_id, order_time
    from ppd_customer_orders
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
),
prep as (
select so.order_id,so.order_time,runner_id,pickup_time,AGE(pickup_time,order_time) as prep_time
    from successfull_orders so
    left join ppd_runner_orders pro
    on so.order_id = pro.order_id
),
num_of_orders_tbl as (
    select order_id,count(*) as num_of_pizzas
    from ppd_customer_orders
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
    group by order_id
)
select corr(EXTRACT(EPOCH FROM prep_time)/ 60,num_of_pizzas) as correlation_between_prep_time_and_num_of_pizzas
from prep p
inner join num_of_orders_tbl notbl
on p.order_id = notbl.order_id;
```

	correlation_between_prep_time_and_num_of_pizzas double precision
1	0.8356841091611149

```sql
--4. What was the average distance travelled for each customer?

select customer_id, ROUND(AVG(distance)::decimal,2) as Avg_distance_travelled
from ppd_customer_orders pco
inner join ppd_runner_orders pro
on pco.order_id = pro.order_id
where cancellation is null
group by customer_id;
```

	customer_id integer	avg_distance_travelled numeric
1	101	20.00
2	102	16.73
3	105	25.00
4	104	10.00
5	103	23.40

```sql
--5. What was the difference between the longest and shortest delivery times for all orders?

select MAX(duration) - MIN(duration) as diff_delivery_times
from ppd_runner_orders;
```

	diff_delivery_times integer
1	30

```sql
--6. What was the average speed for each runner for each delivery and do you notice any trend for these values?

select order_id,runner_id,
ROUND(((distance * 60)/duration)::decimal,2) as speed
from ppd_runner_orders
where cancellation is null;
```

	order_id integer	runner_id integer	speed numeric
1	1	1	37.50
2	2	1	44.44
3	3	1	40.20
4	4	2	35.10
5	5	3	40.00
6	7	2	60.00
7	8	2	93.60
8	10	1	60.00

```sql
--7. What is the successful delivery percentage for each runner?

select runner_id,(100*SUM(
CASE
    when cancellation is null then 1
    else 0
end)/count(*)) as successful_delivery_percentage
from ppd_runner_orders
group by runner_id;
```

	runner_id integer	successful_delivery_percentage bigint
1	3	50
2	2	75
3	1	100

Ingredient Optimisation

```sql
--1. What are the standard ingredients for each pizza?

with ppd_pizza_recipes as (
    select pizza_id,
    trim(
            unnest(
                string_to_array(toppings,',')
            )
        )::integer as topping_id
    from pizza_recipes
)
select pizza_id,
    array_to_string(ARRAY_AGG(topping_name),',') as topping_name
from ppd_pizza_recipes ppdpr
inner join pizza_toppings pt
on ppdpr.topping_id = pt.topping_id
group by pizza_id
order by pizza_id;
```

	pizza_id integer	topping_name text
1	1	Bacon,BBQ Sauce,Beef,Cheese,Chicken,Mushrooms,Pepperoni,Salami
2	2	Cheese,Mushrooms,Onions,Peppers,Tomatoes,Tomato Sauce

```sql
--2. What was the most commonly added extra?

with eti as (
    select trim(unnest(string_to_array(extras,',')))::integer as extras_topping_id
    from ppd_customer_orders
    where extras is not null
)
select eti.extras_topping_id,topping_name,COUNT(*) as no_of_times_ordered_as_extras
from eti
inner join pizza_toppings pt
on eti.extras_topping_id = pt.topping_id
group by eti.extras_topping_id,topping_name
order by no_of_times_ordered_as_extras DESC
FETCH FIRST 1 ROW ONLY;
```

	extras_topping_id integer	topping_name text	no_of_times_ordered_as_extras bigint
1	1	Bacon	4

```sql
--3. What was the most common exclusion?

with eti as (
    select trim(unnest(string_to_array(exclusions,',')))::integer as exclusions_topping_id
    from ppd_customer_orders
    where exclusions is not null
)
select eti.exclusions_topping_id,topping_name,COUNT(*) as no_of_times_excluded
from eti
inner join pizza_toppings pt
on eti.exclusions_topping_id = pt.topping_id
group by eti.exclusions_topping_id,topping_name
order by no_of_times_excluded DESC
FETCH FIRST 1 ROW ONLY;
```

	exclusions_topping_id integer	topping_name text	no_of_times_excluded bigint
1	4	Cheese	4

```sql
--4. Generate an order item for each record in the customers_orders table in the format of one of the following:
--   Meat Lovers
--   Meat Lovers - Exclude Beef
--   Meat Lovers - Extra Bacon
--   Meat Lovers - Exclude Cheese, Bacon - Extra Mushroom, Peppers

with ppd_customer_orders_rn as (
    select *,ROW_NUMBER() OVER(
    PARTITION BY order_id
    ORDER BY order_time) as r_num
from ppd_customer_orders
)
,id_tbl as (
    select order_id,customer_id,pizza_id,
            trim(unnest(string_to_array(exclusions,',')))::int as exclusions,
            trim(unnest(string_to_array(extras,',')))::int as extras,
            order_time,r_num
    from ppd_customer_orders_rn
        UNION all
    select order_id,customer_id,pizza_id,exclusions::int,extras::int, order_time,r_num
    from ppd_customer_orders_rn
    where exclusions is null and extras is null
    order by order_id
)
,name_tbl as (
    select id_tbl.order_id,id_tbl.customer_id,pizza_name, pt.topping_name as exclusions_name, pt1.topping_name,r_num
    from id_tbl
    inner join pizza_names pn
        on id_tbl.pizza_id = pn.pizza_id
    left join pizza_toppings pt
        on id_tbl.exclusions = pt.topping_id
    left join pizza_toppings pt1
        on id_tbl.extras = pt1.topping_id
    order by order_id
)
,agg_tbl as (
select order_id,customer_id,pizza_name,r_num,
    array_to_string(ARRAY_AGG(exclusions_name),', ') as exclusions_name,
    array_to_string(ARRAY_AGG(topping_name),', ') as topping_name
from name_tbl
group by order_id,customer_id,pizza_name,r_num
order by order_id
)
```

```sql
select order_id,customer_id,
    CASE
        when exclusions_name LIKE '' and topping_name LIKE '' then pizza_name
        when exclusions_name NOT LIKE '' and topping_name LIKE '' then CONCAT(pizza_name,' - Exclude ',exclusions_name)
        when exclusions_name LIKE '' and topping_name NOT LIKE '' then CONCAT(pizza_name,' - Extra ',topping_name)
        when exclusions_name NOT LIKE '' and topping_name NOT LIKE '' then CONCAT(pizza_name,' - Exclude ',exclusions_name,' - Extra ',topping_name)
    END as order_description
FROM agg_tbl;
```

	order_id integer	customer_id integer	order_description text
1	1	101	Meatlovers
2	2	101	Meatlovers
3	3	102	Meatlovers
4	3	102	Vegetarian
5	4	103	Meatlovers - Exclude Cheese
6	4	103	Meatlovers - Exclude Cheese
7	4	103	Vegetarian - Exclude Cheese
8	5	104	Meatlovers - Extra Bacon
9	6	101	Vegetarian
10	7	105	Vegetarian - Extra Bacon
11	8	102	Meatlovers
12	9	103	Meatlovers - Exclude Cheese - Extra Bacon, Chicken
13	10	104	Meatlovers - Exclude Mushrooms, BBQ Sauce - Extra Cheese, Bacon
14	10	104	Meatlovers

```sql
--5. Generate an alphabetically ordered comma separated ingredient list for each pizza order from the
--customer_orders table and add a 2x in front of any relevant ingredients
--For example: "Meat Lovers: 2xBacon, Beef, ... , Salami"

with ppd_customer_orders_rn as (
    select *,ROW_NUMBER() OVER(
    PARTITION BY order_id
    ORDER BY order_time) as r_num
from ppd_customer_orders
)
,arr_tbl as (
    select order_id,
        customer_id,
        pp.pizza_id,
        string_to_array(exclusions,', ')::int[] as exclusions,
        string_to_array(extras,', ')::int[] as extras,
        string_to_array(pr.toppings,', ')::int[] as std_ing,
        order_time,
        r_num
    from ppd_customer_orders_rn pp
    left join pizza_recipes pr
    on pp.pizza_id = pr.pizza_id
    order by order_id
)
,ing_ext_tbl as (
    select *,std_ing || extras as ing_extras
    from arr_tbl
)
, ing_ext_exc_tbl as (
    select *,
    CASE
        when exclusions is not null then ARRAY( SELECT x FROM unnest(ing_extras) as x WHERE NOT x = ANY(exclusions))
        else ing_extras
    end as result_ing
    FROM ing_ext_tbl
)
, ing_tbl as (
    select order_id,customer_id,pizza_id,r_num,unnest(result_ing) as ing
    from ing_ext_exc_tbl
)
, freq_top as (
    select order_id,customer_id,pizza_name,topping_name,r_num,count(*) as freq_topping
    from ing_tbl itbl
    left join pizza_names pn
        ON itbl.pizza_id = pn.pizza_id
    left join pizza_toppings pt
        on itbl.ing = pt.topping_id
    group by order_id,customer_id,pizza_name,topping_name,r_num
    order by order_id,topping_name
)
,topp_n_freq as (
    select *,
    CASE
        when freq_topping > 1 then concat(freq_topping,'x',topping_name)
        else topping_name
    end as topp_freq
    from freq_top
)
select order_id,customer_id,pizza_name,r_num,array_to_string(ARRAY_AGG(topp_freq),', ')
from topp_n_freq
group by order_id,customer_id,pizza_name,r_num;
```

	order_id integer	customer_id integer	pizza_name text	r_num bigint	array_to_string text
1	1	101	Meatlovers	1	Bacon, Cheese, Chicken, Mushrooms, Pepperoni, Salami, BBQ Sauce, Beef
2	2	101	Meatlovers	1	Bacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
3	3	102	Meatlovers	2	Bacon, Pepperoni, BBQ Sauce, Salami, Beef, Cheese, Mushrooms, Chicken
4	3	102	Vegetarian	1	Onions, Cheese, Mushrooms, Peppers, Tomato Sauce, Tomatoes
5	4	103	Meatlovers	1	Beef, Mushrooms, Bacon, Salami, Pepperoni, BBQ Sauce, Chicken
6	4	103	Meatlovers	2	Mushrooms, Bacon, BBQ Sauce, Beef, Chicken, Pepperoni, Salami
7	4	103	Vegetarian	3	Peppers, Mushrooms, Tomatoes, Onions, Tomato Sauce
8	5	104	Meatlovers	1	2xBacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
9	6	101	Vegetarian	1	Cheese, Mushrooms, Onions, Peppers, Tomato Sauce, Tomatoes
10	7	105	Vegetarian	1	Bacon, Cheese, Mushrooms, Onions, Peppers, Tomato Sauce, Tomatoes
11	8	102	Meatlovers	1	Bacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
12	9	103	Meatlovers	1	2xBacon, BBQ Sauce, Beef, 2xChicken, Mushrooms, Pepperoni, Salami
13	10	104	Meatlovers	1	2xBacon, Beef, 2xCheese, Chicken, Pepperoni, Salami
14	10	104	Meatlovers	2	Cheese, Pepperoni, Chicken, Bacon, BBQ Sauce, Mushrooms, Beef, Salami

```
--6. What is the total quantity of each ingredient used in all delivered pizzas sorted by most frequent first?

with ppd_customer_orders_rn as (
    select *,ROW_NUMBER() OVER(
    PARTITION BY order_id
    ORDER BY order_time) as r_num
from ppd_customer_orders
)
,arr_tbl as (
    select order_id,
        customer_id,
        pp.pizza_id,
        string_to_array(exclusions,', ')::int[] as exclusions,
        string_to_array(extras,', ')::int[] as extras,
        string_to_array(pr.toppings,', ')::int[] as std_ing,
        order_time,
        r_num
    from ppd_customer_orders_rn pp
    left join pizza_recipes pr
    on pp.pizza_id = pr.pizza_id
    order by order_id
)
,ing_ext_tbl as (
    select *,std_ing || extras as ing_extras
    from arr_tbl
)
```

```sql
, ing_ext_exc_tbl as (
    select *,
    CASE
        when exclusions is not null then ARRAY( SELECT x FROM unnest(ing_extras) as x WHERE NOT x = ANY(exclusions))
        else ing_extras
    end as result_ing
    FROM ing_ext_tbl
)
, ing_tbl as (
    select order_id,customer_id,pizza_id,r_num,unnest(result_ing) as ing
    from ing_ext_exc_tbl
)
, final as (
    select topping_name,count(*) as freq_topping
    from ing_tbl itbl
    left join pizza_names pn
        ON itbl.pizza_id = pn.pizza_id
    left join pizza_toppings pt
        on itbl.ing = pt.topping_id
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
    group by topping_name
)
select *
from final
order by freq_topping DESC;
```

	topping_name text	freq_topping bigint
1	Bacon	12
2	Mushrooms	11
3	Cheese	10
4	Pepperoni	9
5	Chicken	9
6	Salami	9
7	Beef	9
8	BBQ Sauce	8
9	Tomato Sauce	3
10	Onions	3
11	Tomatoes	3
12	Peppers	3

Pricing and Ratings

```sql
--1. If a Meat Lovers pizza costs $12 and Vegetarian costs $10 and there
-- were no charges for changes - how much money has Pizza Runner made so far if there are no delivery fees?

with freq_pizza as (
    select pco.pizza_id,pizza_name,count(*) as freq_pizza
    from ppd_customer_orders pco
    inner join pizza_names pn
    on pco.pizza_id = pn.pizza_id
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
    group by pco.pizza_id,pizza_name
)
, capital as (
    select *,
        CASE
            when pizza_name = 'Meatlovers' then freq_pizza * 12
            when pizza_name = 'Vegetarian' then freq_pizza * 10
        END as capital_earned
    from freq_pizza
)
select sum(capital_earned) as total_amount_earned
from capital
```

	total_amount_earned numeric
1	138

```sql
--2. What if there was an additional $1 charge for any pizza extras?
-- Add cheese is $1 extra

with ppd_customer_orders_rn as (
    select *,ROW_NUMBER() OVER(
    PARTITION BY order_id
    ORDER BY order_time) as r_num
from ppd_customer_orders
)
,arr_tbl as (
    select order_id,
        customer_id,
        pp.pizza_id,
        string_to_array(exclusions,', ')::int[] as exclusions,
        string_to_array(extras,', ')::int[] as extras,
        string_to_array(pr.toppings,', ')::int[] as std_ing,
        order_time,
        r_num
    from ppd_customer_orders_rn pp
    left join pizza_recipes pr
    on pp.pizza_id = pr.pizza_id
    order by order_id
)
,ing_ext_tbl as (
    select *,std_ing || extras as ing_extras
    from arr_tbl
)
, ing_ext_exc_tbl as (
    select *,
    CASE
        when exclusions is not null then ARRAY( SELECT x FROM unnest(ing_extras) as x WHERE NOT x = ANY(exclusions))
        else ing_extras
    end as result_ing
    FROM ing_ext_tbl
)
, ing_tbl as (
    select order_id,customer_id,pizza_id,r_num,unnest(result_ing) as ing
    from ing_ext_exc_tbl
)
,is_chs as (
    select order_id,customer_id,itbl.pizza_id,r_num,pizza_name,
        CASE
            when topping_name = 'Cheese' then 1
            else 0
        END as is_cheese
    from ing_tbl itbl
    left join pizza_names pn
        ON itbl.pizza_id = pn.pizza_id
    left join pizza_toppings pt
        on itbl.ing = pt.topping_id
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
)
, ext_chs as (
    select order_id,customer_id,pizza_id,pizza_name,r_num,sum(is_cheese) as extra_cheese_price
    from is_chs
    group by order_id,customer_id,pizza_id,pizza_name,r_num
)
, price as (
    select pizza_name,count(*) as freq_orders,SUM(extra_cheese_price) as ext_cheese_price
    from ext_chs
    group by pizza_name
)
, final_tbl as (
    select *,
        CASE
            when pizza_name = 'Meatlovers' then freq_orders*12 + ext_cheese_price
            when pizza_name = 'Vegetarian' then freq_orders*10 + ext_cheese_price
        end as capital_earned
    from price
)
select sum(capital_earned) as total_amount_earned
from final_tbl;
```

	total_amount_earned numeric
1	148

```sql
--3. The Pizza Runner team now wants to add an additional ratings system that
-- allows customers to rate their runner, how would you design an additional table for this
-- new dataset - generate a schema for this new table and insert your own data for ratings
-- for each successful customer order between 1 to 5.

CREATE TABLE ratings (
    order_id integer,
    rating integer
);

INSERT INTO ratings
VALUES (1,3),
    (2,4),
    (3,5),
    (4,4),
    (5,3),
    (7,1),
    (8,4),
    (10,5);

select * from ratings;
```

	order_id integer	rating integer
1	1	3
2	2	4
3	3	5
4	4	4
5	5	3
6	7	1
7	8	4
8	10	5

```sql
--4. Using your newly generated table - can you join all of the information together
--to form a table which has the following information for successful deliveries?
-- customer_id
-- order_id
-- runner_id
-- rating
-- order_time
-- pickup_time
-- Time between order and pickup    o
-- Delivery duration
-- Average speed
-- Total number of pizzas

with orders as (
    select order_id,customer_id,order_time,count(*) as tot_num_of_pizzas
    from ppd_customer_orders
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
    group by order_id,customer_id,order_time
)
select customer_id,
    o.order_id,
    runner_id,
    rating,
    order_time,
    pickup_time,
    (pickup_time - order_time) as Time_between_order_and_pickup,
    duration as Delivery_duration,
    ROUND(((distance * 60)/duration)::decimal,2) as Speed,
    tot_num_of_pizzas as Total_number_of_pizzas
from orders o
left join ppd_runner_orders pro
    on o.order_id = pro.order_id
left join ratings r
    on o.order_id = r.order_id;
```

	customer_id integer	order_id integer	runner_id integer	rating integer	order_time timestamp without time zone	pickup_time timestamp without time zone	time_between_order_and_pickup interval	delivery_duration integer	speed numeric	total_number_of_pizzas bigint
1	101	1	1	3	2020-01-01 18:05:02	2020-01-01 18:15:34	00:10:32	32	37.50	1
2	101	2	1	4	2020-01-01 19:00:52	2020-01-01 19:10:54	00:10:02	27	44.44	1
3	102	3	1	5	2020-01-02 23:51:23	2020-01-03 00:12:37	00:21:14	20	40.20	2
4	103	4	2	4	2020-01-04 13:23:46	2020-01-04 13:53:03	00:29:17	40	35.10	3
5	104	5	3	3	2020-01-08 21:00:29	2020-01-08 21:10:57	00:10:28	15	40.00	1
6	105	7	2	1	2020-01-08 21:20:29	2020-01-08 21:30:45	00:10:16	25	60.00	1
7	102	8	2	4	2020-01-09 23:54:33	2020-01-10 00:15:02	00:20:29	15	93.60	1
8	104	10	1	5	2020-01-11 18:34:49	2020-01-11 18:50:20	00:15:31	10	60.00	2

```sql
--5. If a Meat Lovers pizza was $12 and Vegetarian $10 fixed prices with
-- no cost for extras and each runner is paid $0.30 per kilometre traveled -
-- how much money does Pizza Runner have left over after these deliveries?


with pizza_price as (
    select *,
    CASE
        when pizza_id = 1 then 12
        when pizza_id = 2 then 10
    END as price
    from ppd_customer_orders pco
    where order_id not in (select order_id from ppd_runner_orders where cancellation is not null)
)
,order_price as (
    select order_id,sum(price) as pizza_cost
    from pizza_price
    group by order_id
),
del_cost as (
    select *,(distance * 0.3) as delivery_cost
    from order_price op
    left join ppd_runner_orders pro
    on op.order_id = pro.order_id
)
select ROUND(SUM(pizza_cost - delivery_cost)::decimal,2) as net_profit
from del_cost;
```

	net_profit numeric
1	94.44

```sql
--Bonus Questions
-- If Danny wants to expand his range of pizzas -
-- how would this impact the existing data design? Write an INSERT statement
-- to demonstrate what would happen if a new Supreme pizza with all
-- the toppings was added to the Pizza Runner menu?

INSERT INTO pizza_recipes
VALUES (3,'1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12');
```